

UNIVERSITY OF SCIENCE AND TECHNOLOGY CHITTAGONG (USTC)



Lab report-01

Department: Computer Science & Engineering
Course Title: Computer Graphics and Image Processing
Course Code: CSE 417

<i>Submitted By</i>	<i>Submitted To</i>
Name: Bijon Chowdhury Prem Reg. No: 23070165 Roll No: 1071 Batch: CSE 41 (B) Semester: 7 th	Name: Umme Aiman Jannat Designation: Lecturer CSE, FSET, USTC

Submission Date: 27-08-2026

Topic: 2D Transformation

Experiment No.: 1

Experiment Title: Translation of a 2D Image Objective

To shift an image horizontally and vertically using the translation transformation in OpenCV.

Requirement

- Python 3
- OpenCV (cv2)
- NumPy
- Matplotlib
- Input image: image.jpg
- Jupyter Notebook / Google Colab / Python IDE

Basics

Translation is an affine transformation that moves each pixel of an image by a specified number of pixels along the x-axis and y-axis.

The translation matrix is $M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$, where t_x is the horizontal shift and t_y is the vertical shift. For this experiment, $t_x = 100$ pixels and $t_y = 50$ pixels. The tree image is shifted 100 pixels to the right and 50 pixels downward.

Procedure

1. Import the required libraries.
2. Read the tree image using OpenCV.
3. Convert the image from BGR to RGB format.
4. Obtain the image height and width.
5. Define translation values.
6. Create the translation matrix.
7. Apply translation using `cv2.warpAffine()`.
8. Display the original and translated images.

Source Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

h, w =
image.shape[:2] tx,
ty = 100, 50

M = np.float32([[1, 0, tx],
[0, 1, ty]])

translated = cv2.warpAffine(image, M, (w, h))

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Tree Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(translated)
plt.title("Translated Image (tx=100, ty=50)")
plt.axis("off")
plt.show()
```

Output

Original Tree Image



Translated Image (tx=100, ty=50)



Figure 1: Original tree image and translated tree image.

Result

The tree image was successfully translated by 100 pixels in the x-direction and 50 pixels in the y-direction.

Conclusion

Translation changes the location of an image without changing its size, shape, or orientation.

Experiment No.: 2

Experiment Title: Scaling of a 2D Image Objective

To scale a 2D image by applying different scaling factors along the horizontal and vertical directions.

Requirement

- Python 3
- OpenCV (cv2)
- NumPy
- Matplotlib
- Input image: image.jpg
- Jupyter Notebook / Google Colab / Python IDE

Basics

Scaling is a transformation that changes the size of an image. The image may be enlarged or reduced depending on the scaling values.

The scaling matrix is $M = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \end{bmatrix}$. Here $sx = 1.5$ and $sy = 2.2$, so the width is enlarged by 1.5 times and the height by 2.2 times.

Procedure

1. Import OpenCV, NumPy, and Matplotlib libraries.
2. Read the tree image.
3. Convert the image from BGR format to RGB format.
4. Obtain the image height and width.
5. Define $sx = 1.5$ and $sy = 2.2$.
6. Create the scaling matrix.
7. Apply scaling using `cv2.warpAffine()`.
8. Display the original and scaled images.

Source Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

h, w =
image.shape[:2]
sx, sy = 1.5,
2.2

M = np.float32([[sx, 0, 0],
[0, sy, 0]])

scaled = cv2.warpAffine(image, M, (int(w*sx),
int(h*sy))) plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Tree
Image") plt.axis("off")

plt.subplot(1,
2, 2)
plt.imshow(scal
ed)
plt.title("Scaled Image (sx=1.5, sy=2.2)")
plt.axis("off")
plt.show()
```

Output

Original Tree Image



Scaled Image (sx=1.5, sy=2.2)



Figure 2: Original tree image and scaled tree image with $sx = 1.5$ and $sy = 2.2$.

Result

The tree image was successfully scaled with a horizontal factor of 1.5 and a vertical factor of 2.2.

Conclusion

Scaling changes image dimensions. Since the vertical factor is greater than the horizontal factor, the image appears stretched vertically.

Experiment No.: 3

Experiment Title: Rotation of a 2D Image Objective

To rotate a 2D image by 45 degrees about its center using OpenCV.

Requirement

- Python 3
- OpenCV (cv2)
- Matplotlib
- Input image: image.jpg
- Jupyter Notebook / Google Colab / Python IDE

Basics

Rotation is a geometric transformation in which an image is turned around a fixed point. Usually, the center of the image is used as the rotation point.

OpenCV provides **cv2.getRotationMatrix2D(center, angle, scale)**. In this experiment, the image is rotated by 45 degrees around its center.

Procedure

1. Import the required libraries.
2. Read the tree image.
3. Convert the image from BGR to RGB.
4. Determine the image center.
5. Create a rotation matrix for 45 degrees.
6. Apply rotation using cv2.warpAffine().
7. Display the original and rotated images.

Source Code

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

h, w =
image.shape[:2]
center = (w //
2, h // 2)
angle = 45

M = cv2.getRotationMatrix2D(center,
angle, 1.0) rotated =
cv2.warpAffine(image, M, (w, h))

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original
Tree Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(rotated)
plt.title("Rotated
Image - 45°")
plt.axis("off")
plt.show()
```

Output

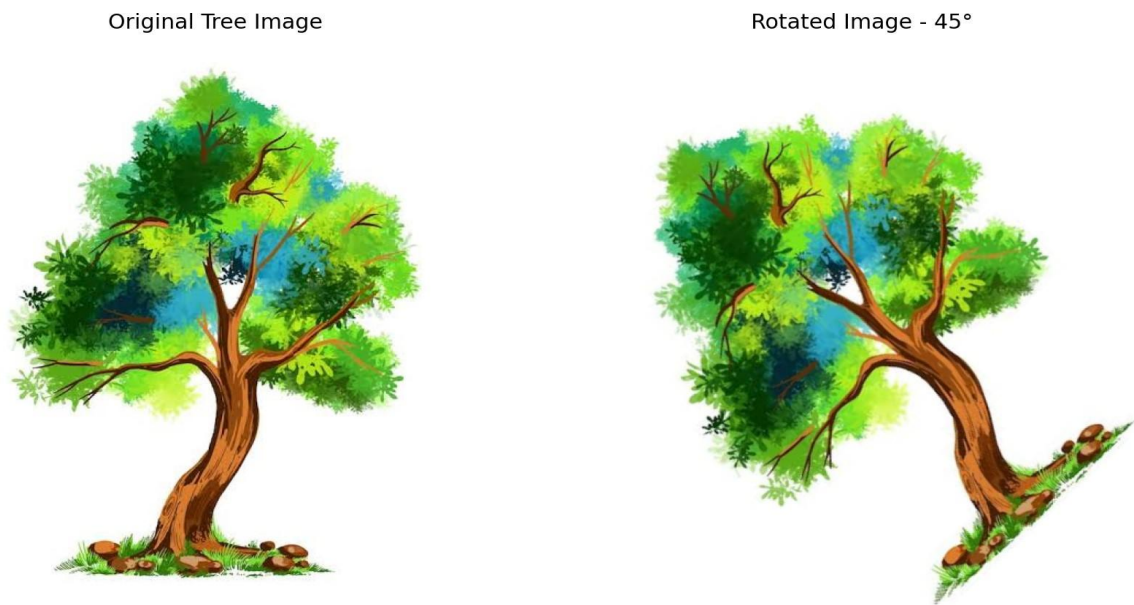


Figure 3: Original tree image and tree image rotated by 45 degrees.

Result

The tree image was successfully rotated by 45 degrees around its center.

Conclusion

Rotation changes the orientation of an image while maintaining the output canvas size; blank regions may occur around the corners.

Experiment No.: 4

Experiment Title: Reflection About X-Axis Objective

To reflect a 2D image about the X-axis using OpenCV.

Requirement

- Python 3
- OpenCV (cv2)
- Matplotlib
- Input image: image.jpg
- Jupyter Notebook / Google Colab / Python IDE

Basics

Reflection creates a mirror image of an object. Reflection about the X-axis flips the image vertically. The upper portion moves to the lower portion, and the lower portion moves to the upper portion.

In OpenCV, vertical reflection is performed using **cv2.flip(image, 0)**.

Procedure

1. Import the required libraries.
2. Read the tree image.
3. Convert the image from BGR to RGB.
4. Apply vertical flipping using **cv2.flip(image, 0)**.
5. Display the original and reflected images.

Source Code

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg')
image = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB) reflected =

cv2.flip(image, 0)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Tree
Image") plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(reflected)
plt.title("Reflection About X-
axis") plt.axis("off")
plt.show()
```

Output

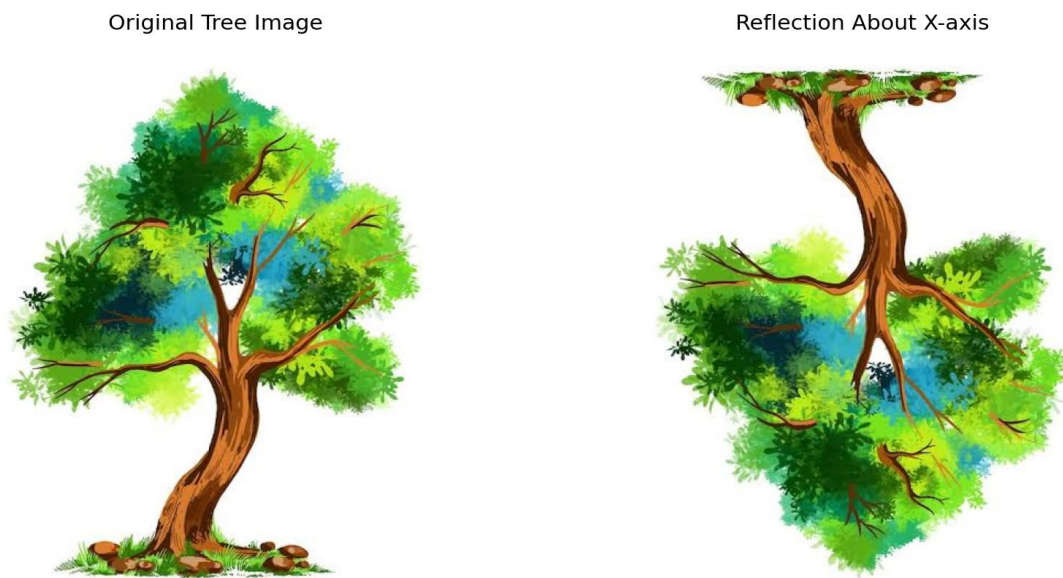


Figure 4: Original tree image and reflection about the X-axis.

Result

The tree image was successfully reflected about the X-axis.

Conclusion

Reflection about the X-axis produces a vertical mirror image or upside-down version of the original image.

Experiment No.: 5

Experiment Title: Reflection About Y-Axis Objective

To reflect a 2D image about the Y-axis using OpenCV.

Requirement

- Python 3
- OpenCV (cv2)
- Matplotlib
- Input image: image.jpg
- Jupyter Notebook / Google Colab / Python IDE

Basics

Reflection about the Y-axis flips an image horizontally. The left side moves to the right side, and the right side moves to the left side.

In OpenCV, horizontal reflection is performed using **cv2.flip(image, 1)**.

Procedure

1. Import the required libraries.
2. Read the tree image.
3. Convert the image from BGR to RGB.
4. Apply horizontal flipping using **cv2.flip(image, 1)**.
5. Display the original and reflected images.

Source Code

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

reflected = cv2.flip(image, 1)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Tree Image") plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(reflected)
plt.title("Reflection About Y-axis") plt.axis("off")
plt.show()
```

Output

Original Tree Image



Reflection About Y-axis



Figure 5: Original tree image and reflection about the Y-axis.

Result

The tree image was successfully reflected about the Y-axis.

Conclusion

Reflection about the Y-axis generates a horizontal mirror image of the original tree image.

Experiment No.: 6

Experiment Title: Combined Transformation of a 2D Image Objective

To apply scaling, rotation, and translation transformations sequentially to a 2D image using OpenCV.

Requirement

- Python 3
- OpenCV (cv2)
- NumPy
- Matplotlib
- Input image: image.jpg
- Jupyter Notebook / Google Colab / Python IDE

Basics

A combined transformation applies more than one transformation to an image. Three transformations are applied in sequence: **Scaling** by 1.2 in both directions, **Rotation** by 30 degrees, and **Translation** by 50 pixels right and 30 pixels downward.

Transformation flow: **Original Tree Image** → **Scaling** → **Rotation** → **Translation** → **Final Image**.

Procedure

1. Import the required libraries.
2. Read the tree image.
3. Convert the image from BGR to RGB.
4. Scale the image by 1.2 in both directions.
5. Find the center of the scaled image.
6. Rotate the scaled image by 30 degrees.
7. Create a translation matrix using tx = 50 and ty = 30.
8. Apply translation to the rotated image.
9. Display the original, rotated, and final transformed images.

Source Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Scaling
scaled = cv2.resize(image, None, fx=1.2, fy=1.2)

# Rotation
h2, w2 = scaled.shape[:2]
center = (w2 // 2, h2 // 2)
M_rotation = cv2.getRotationMatrix2D(center, 30, 1.0)
rotated = cv2.warpAffine(scaled, M_rotation, (w2, h2))

# Translation
tx, ty = 50, 30
M_translation = np.float32([[1, 0, tx],
[0, 1, ty]])
transformed = cv2.warpAffine(rotated, M_translation, (w2+tx, h2+ty))

plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(image)
plt.title("Original Tree Image")
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
plt.imshow(rotated)
plt.title("After Scaling and Rotation")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(transformed)
plt.title("Final Transformed Image")
plt.axis("off")
plt.show()
```

Output

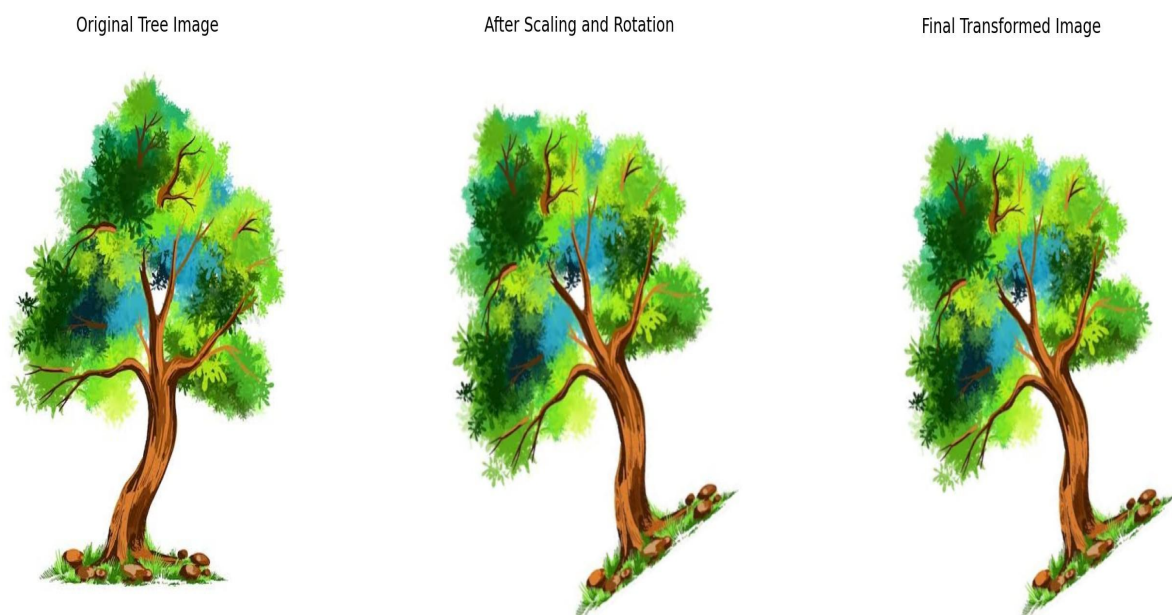


Figure 6: Combined transformation using scaling, rotation, and translation.

Result

The tree image was successfully scaled by 1.2, rotated by 30 degrees, and translated by 50 pixels horizontally and 30 pixels vertically.

Conclusion

Combined transformations are useful for performing multiple geometric operations on an image. OpenCV provides efficient functions for resizing, rotating, translating, reflecting, and displaying digital images.